AGRISCAN AI SOLUTION STEPS

Start

1. Download the PlantVillage dataset

Download the PlantVillage dataset from Huggingface

Unzip the folder to the project directory

Ensure the dataset folder structure with train and test subdirectories

Store this folder in the project root directory alongside the Python script

Make sure the folders are sorted in alphabetical order

2. Set up the development environment

Create a virtual environment (venv)

Activate the virtual environment

Install required dependencies: tensorflow, keras, numpy, matplotlib, pillow, scikit-learn

3. Import necessary libraries

Import os, json, argparse, random, numpy, tensorflow, matplotlib

Import PIL, keras models, layers, utils, metrics, optimizers, callbacks

Import sklearn metrics and class_weight

Import disease_knowledge_base for disease information

4. Set global constants

Set SEED = 2025 for reproducibility

Set random, numpy, and TensorFlow seeds

Set AUTOTUNE = tf.data.AUTOTUNE for performance

Set IMAGE_SIZE = (128, 128) for resizing images

Set BATCH_SIZE = 64 for training batch size

5. Read the data

Create constant variables for directory paths: BASE_DIRECTORY, TRAIN_DIRECTORY, TEST_DIRECTORY

Validate that all directories exist

Create a list to house the class names in alphabetical order

Use Keras image_dataset_from_directory to load train, validation and test datasets

Ensure test directory has same classes as train directory

Use labels="inferred" and label_mode="categorical" for multi-class classification

Set image_size=IMAGE_SIZE and batch_size=BATCH_SIZE

Set shuffle=True for training, shuffle=False for validation/test

6. Prepare the images

Create rescaling layer to normalize pixel values from [0, 255] to [0, 1]

Create augmentation layer with random flip, zoom and contrast

Apply augmentation to training data only

Apply rescaling to all datasets (train, validation, test)

Use .map() to apply transformations

Use .cache() and .prefetch(AUTOTUNE) for performance

7. Build the CNN model

Create Sequential model with name "agriscan_cnn"

Add Input layer with shape (128, 128, 3)

Create 4 convolutional blocks with increasing filters (32, 64, 128, 256)

Each block: Conv2D + MaxPooling2D

Add Flatten layer to convert 2D to 1D

Add Dense layer with 256 neurons and ReLU activation

Add Dropout layer with rate 0.5

Add Output Dense layer with Softmax activation

Compile model with Adam optimizer, categorical_crossentropy loss, and metrics

8. Calculate class weights

Extract labels from training dataset

Use sklearn's compute_class_weight with class_weight='balanced'

Create dictionary mapping class indices to weights

Handle dataset imbalance where some diseases have fewer examples

9. Set up training callbacks

Create EarlyStopping callback (monitor val_loss, patience=3)

Create ModelCheckpoint callback (save best model based on val_accuracy)

Create ReduceLROnPlateau callback (reduce learning rate on plateau)

10. Train the model

Create models/ and results/ directories

Load datasets, build model, calculate class weights

Save class names and class weights to JSON files

Train model with class weights and callbacks

Save training history to JSON

Generate and save training history plots

Load best model checkpoint for evaluation

11. Evaluate the model

Evaluate model on test dataset

Print test metrics: loss, accuracy, precision, recall

Generate predictions on test dataset

Generate confusion matrix and save visualization

Generate classification report and save to JSON

12. Create prediction function

Create validate_path() function for file path validation

Create load_model() function to load trained model and class names

Create predict_image() function supporting multiple input types

Handle different image modes and convert to RGB

Resize images and normalize pixel values

Generate predictions and get top 3 predictions

Retrieve disease information from knowledge base

Determine reliability level based on confidence score

Return results dictionary with all prediction information

13. Create command-line interface

Create predict_cli() function for CLI interface

Add argparse for command-line argument parsing

Create arguments: --train-class-weighted, --predict, --epochs

Handle interactive mode when no image path provided

Display results in formatted manner with disease information

Show special warning for low-confidence predictions

14. Create visualization functions

Create plot_training_history() function

Generate loss and accuracy curves

Use subplots for side-by-side display

Save as high-resolution PNG

15. Create main function

Create main() function to orchestrate the program

Set up argument parser with description

Parse arguments and handle training/prediction modes

Return appropriate exit code

16. Add if __name__ == "__main__" guard

Execute main() function when script is run directly

17. Test the system

Train the model with --train-class-weighted flag

Verify model and results are saved

Test prediction with sample image

Test interactive mode

18. Document the code

Add docstrings to all functions

Create documentation, poster, flowchart and steps files